

AETHER BREAK

A small, skill-forward 3D browser arena fighter

Software Requirements Specification & Technical Architecture

DOCUMENT STATUS	Concept baseline — ready for technical spike
VERSION	1.0
PREPARED	31 July 2026
PRODUCT POSITION	Original competitive game concept; no licensed Street Fighter assets, names, or moves

DECISION IN ONE LINE Build a stylised 2.5D fighter with 3D characters and arenas—not a free-roam brawler—so guns, magic, supers, and online play remain readable, performant, and achievable for a small team.

This SRS defines the launchable MVP and its upgrade path. “MUST” requirements form the release baseline; “SHOULD” requirements are planned only after the vertical slice validates combat feel and browser performance.

Document map

A practical specification for a browser-first original fighter: product intent, player experience, formal requirements, technical design, delivery plan, and launch gates.

SECTION	WHAT IT ANSWERS
1. Product definition	Vision, principles, players, scope, success measures
2. Game design specification	Match loop, controls, resources, roster, modes, arenas
3. Functional requirements	Testable requirements and acceptance criteria
4. Non-functional requirements	Performance, compatibility, reliability, accessibility, security
5. Recommended stack & decisions	Tools selected, alternatives, repository conventions
6. System architecture	Client, backend, online simulation, APIs, data and asset flow
7. Content & production pipeline	Art, animation, audio, balance data, authoring
8. Quality, security & operations	Testing, anti-cheat, privacy, telemetry, release
9. Delivery plan & risk register	Milestones, staffing assumptions, launch readiness
Appendices	Input map, schemas, state machine, glossary

How to use this document

- The MVP is deliberately constrained: four characters, one competitive ruleset, one arena layout with visual variants, local play, bots, and unranked online. This is the smallest package that can prove the core fantasy.
- Keep CombatCore free of rendering and browser APIs. It must run unchanged in the browser and on the match server; this single choice protects fairness and makes rollback feasible.
- Create a playable vertical slice before producing a full roster. A fun 1v1 loop, hit feedback, input buffer, and 60 FPS are higher priority than more abilities or cosmetics.

SCOPE RULE Every new character, arena, projectile type, or game mode must have a performance budget, balance test plan, and content owner before it enters a sprint.

1. Product definition

1.1 Vision and elevator pitch

Aether Break is an original, stylised 3D browser arena fighter for PC. In fast, best-of-three duels, players combine close-range strings, guarded gunfire, character abilities, and a charged ultimate to out-read an opponent. It has the immediate clarity of a classic lane fighter, rendered as a vibrant 3D world with short cinematic flourishes—not long cutscenes that interrupt competition.

PLAYER PROMISE “In ten seconds I know what my character does; in ten matches I discover clever ways to outplay someone.”

1.2 Product principles

PILLAR	DESIGN IMPLICATION
Readability beats spectacle	The camera, silhouettes, hit flashes, sound cues, and VFX must tell players what happened at a glance. Supers may be dramatic but lock control for ≤ 0.8 seconds.
Weapons are verbs, not wins	Guns establish range or combo routes; they use ammo/heat, recovery, and block/punish windows. No hitscan full-screen damage with no counterplay.
Skillful, not inaccessible	Simple universal inputs with optional advanced routes, forgiving input buffer, clear training tools, keyboard and gamepad parity.
Browser-native discipline	Fast first play, compressed assets, WebGL2 fallback, and no mandatory download or launcher.
Small and expandable	Data-driven fighters and abilities; a polished 4-character launch can grow without rewriting the combat engine.

1.3 Target players and rating

PERSONA	NEED	DESIGN RESPONSE
Competitive newcomer	Can I play immediately without memorising 40 commands?	Three normal attacks, two specials, ranged tool, guard, dash, and one ultimate. On-screen move hints and training challenges.
Social browser player	Can friends play in a link without installations?	Guest/local play first; optional account for online identity and progression. A shareable room code is an early online feature.
Fighting-game enthusiast	Will spacing, timing, matchups, and latency feel legitimate?	Frame data in training, input history, deterministic simulation, server-authoritative validation, and post-match network metrics.

Target rating: Teen / PEGI 12–16 equivalent, depending on final regional submission. Combat is stylised energy impact only: no gore, realistic injury, hate content, gambling, or predatory monetisation.

1.4 Goals, non-goals and success measures

GOALS (MVP)	EXPLICIT NON-GOALS (MVP)
A satisfying 1v1 duel at 60 FPS on a mid-range desktop browser.	Open-world exploration, free-roam 3D camera, 8-player modes, or a story campaign.
Four distinct fighters with guns, magic, abilities, and ultimates.	Mobile-first controls, cross-platform native clients, VR, or user-generated content.
Local PvP, training vs. bot, and unranked online in a supported region.	Ranked seasons, esports features, voice chat, clan systems, or battle pass at launch.
A repeatable art/animation/data pipeline for post-launch characters.	Pay-to-win items, loot boxes, gameplay stat purchases, or third-party IP imitation.

MEASURE	VERTICAL SLICE EXIT	MVP LAUNCH TARGET
Combat fun	≥70% of 12 structured testers voluntarily rematch after 15 minutes.	≥45% of first-session players complete 5 matches in usability playtests.
Smoothness	60 FPS median on target test machine in a populated arena.	≥95% of supported-session frames ≤16.7 ms; clear quality fallback below target.
Online integrity	Two remote clients complete 30-minute soak without divergent confirmed state.	≥99.5% completed match sessions record a validated winner or explicit disconnect reason.
Load experience	First interactive practice scene <15 s on 10 Mbps cold cache.	Initial critical bundle/assets <25 MB compressed; arena extras stream afterward.

2. Game design specification

2.1 Match loop and camera

Each match is a side-on 2.5D duel inside a fully 3D arena. Fighters move left/right, jump, crouch, dash, and may use a short, cooldown-gated sidestep lane evade. The camera automatically keeps both fighters framed, never rotates around them, and uses subtle depth/parallax only for presentation. This preserves the readable spacing of a traditional fighter while delivering a 3D indie-game look.

RULE	PROPOSED VALUE	RATIONALE
Match format	Best of 3 rounds; 90 seconds/round	Short browser sessions; enough time for adaptation.
Health	1,000 HP each	Simple balance vocabulary; equal baseline health in MVP.
Flux meter	0–100; gained for dealing, receiving, parrying and objectives	Enables comebacks without rewarding passive stalling.
Ultimate	Available at 100 Flux; one use per round; 250–320 typical damage	High-impact choice, not an automatic round win.
Guard	Hold guard; chip is low; guard crush after sustained pressure	Creates offense/defense decisions and gun counterplay.
Inputs	6-frame input buffer; 3-frame initial online input delay	Accessible feel while remaining deterministic.
Simulation	60 fixed ticks/s; integer/fixed-point core	Supports precise frame data, replay, server simulation, rollback.

2.2 Core verbs and counterplay

VERB	PLAYER ACTION	COUNTERPLAY / RULE
Strike	Light and heavy normals create strings, launches, or knockdown.	Block, backdash, spacing, parry on telegraphed attacks; unsafe heavies are punishable.
Fire	Character-specific gun, bolt, or ranged cast. Ammo/heat or cooldown limits repeated use.	Guard, sidestep, jump, projectile clash or character-specific reflect. Every full-screen tool has recovery.
Ability	Two character-defining specials. One favors neutral, one supports conversion/defense.	Cooldown, resource, startup cue, and a known punish window or positional tradeoff.
Dash / evade	Forward dash for pressure; backdash invulnerability is brief; sidestep escapes a line.	Bait and punish recovery; tracking attacks beat sidestep; throw beats over-blocking.
Ultimate	Spend full Flux on a fast, stylised super that changes the exchange.	No unavoidable round-ending cutscene; ultimate has startup, range/confirm conditions, and damage scaling.

2.3 Combat guardrails

- Core hit detection uses authored hitboxes/hurtboxes and capsule/rectangle overlap in CombatCore—never renderer raycasts or Rapier physics. Every hit is resolved by a deterministic tick and data-driven move definition.
- Ranged attacks are either discrete projectiles with an authored travel rate or short-range hitscan cones. They are not arbitrary pointer-aimed bullets. This keeps keyboard/gamepad parity and makes network validation practical.
- Use hit-stop (typically 4–10 frames), directional hit sparks, short camera shake, and distinct impact audio. Effects are cosmetic; they must not change the authoritative timeline.
- Launch one universal throw, throw-tech window, air recovery rule, and two-stage combo scaling. No infinities: after a defined juggle/chain budget, the target falls into protected knockdown.

2.4 Launch roster — four distinct combat fantasies

FIGHTER	FANTASY & WEAPON	ABILITY 1 / ABILITY 2	ULTIMATE & BALANCE HOOK
Nyra Vex Rift Gunslinger	Agile bounty hunter; twin arc pistols; mid-range harassment. Difficulty: 2/5	Phase Step: short directional blink that leaves a vulnerable exit. Ricochet Sigil: slow orb that bends one shot and creates set play.	Event Horizon: place a rift zone, then a 3-shot crossfire if Nyra confirms a hit. Low health-equivalent defense; limited ammo heat.
Bram Kade Forge Warden	Armoured enforcer; gauntlet, compact scattergun; close-range pressure. Difficulty: 2/5	Bulwark: timed barrier absorbs a single projectile. Furnace Rush: armored shoulder charge, punishable on block.	Last Foundry: overclock gauntlet slam, then a short burn field. Slow walk speed; must earn entry.
Iria Sol Prism Magus	Runic staff and prism cards; zones space with elemental geometry. Difficulty: 4/5	Prism Gate: choose forward/above placement, delayed bolt. Mirror Ward: briefly reflects a non-super projectile.	Sevenfold Star: a confirm-only constellation burst that rewards prepared gates. Fragile at close range; cooldown-led defense.
Kellan Wisp Stormblade	Techno-mystic duelist; energy blade plus coil carbine; rushdown/mobility. Difficulty: 3/5	Volt Tether: line pull on hit or recoil hop on miss. Static Veil: one-hit decoy that drains Flux on activation.	Tempest Divide: mobility super with three directional slashes; whiff leaves long recovery. Low raw damage per hit, high execution.

ROSTER RULE Every fighter must be viable with the universal controls, have one visible “I understand that” counter, and require no more than two unique resources. Character identity comes from spacing and timing—not obscured exceptions.

2.5 Arenas, presentation and modes

ITEM	MVP SPECIFICATION	POST-MVP OPTION
Arena	One non-interactive 2.5D layout: “Skyrail Foundry,” with day/night and weather lighting variants. No stage hazards.	Second layout with a different visual language only after netcode and roster balance are stable.
Camera	Locked side-on follow camera, dynamic zoom bounded by two player positions, reduced-motion option.	Spectator camera after ranked/observer needs are defined.
Modes	Tutorial, training, Versus local, Versus bot, online casual (quick match + room code).	Ranked, seasonal ladders, replay browser, tournament lobbies.

ITEM	MVP SPECIFICATION	POST-MVP OPTION
Progression	Profile level and cosmetic-only colorway/unlock tokens. Guest can play locally and practice.	Account-linked cosmetics and challenges—never gameplay stat boosts.
Bots	Three difficulty presets driven by state-aware intent rules, reaction delay, and deliberate human-like errors.	Learned behavior profiles only if privacy and QA budget permit.

3. Functional requirements

Priority: MUST = release blocking; SHOULD = planned for the MVP window but may move to the first update; COULD = explicitly deferred. All requirements are testable with the acceptance statement below.

3.1 Entry, profile, menus and settings

FR-001 First playable flow

Requirement. The game shall offer Practice as a no-account, no-payment path from the landing screen. Asset download status and browser support errors shall be understandable.

Acceptance. On a clean supported browser, a player can reach practice without signing in; unsupported WebGL2/WebGPU capabilities show a clear fallback message and help link. **MUST**

FR-002 Account and guest identity

Requirement. The game shall support guest local play and optional authenticated profiles for online, progression, and cosmetics. The user can delete their profile through a documented request path.

Acceptance. Two guests can start local Versus; online queue requires a valid authenticated session; profile deletion request is logged and data is removed/anonymised per retention policy. **MUST**

FR-003 Settings and accessibility

Requirement. The game shall persist graphics, sound, input, language, color/flash, and reduced-motion settings locally and, for signed-in users, sync eligible preferences to profile.

Acceptance. A tester can remap every gameplay action, mute each audio bus, enable high contrast/reduced flash, and retain those choices after reload. **MUST**

FR-004 Tutorial onboarding

Requirement. The game shall offer a 4–6 minute interactive tutorial covering movement, attack, block, ranged counterplay, ability, and ultimate.

Acceptance. New player completes six prompted actions and enters a bot match; all prompts are skippable and replayable. **SHOULD**

3.2 Gameplay and match flow

FR-010 Deterministic combat state

Requirement. CombatCore shall advance a fixed 60 Hz simulation using serialisable state, seeded RNG, integer/fixed-point values, and authored move data. It shall not read clock, DOM, renderer, or network APIs.

Acceptance. The same initial state plus 10,000 recorded input frames produces identical state hashes in Node and at least two supported browsers. **MUST**

FR-011 Input system

Requirement. The game shall support keyboard and standard browser gamepad input with remapping, input buffering, simultaneous-opposite-direction handling, and focus-loss safety.

Acceptance. Automated input tests verify mapped actions, six-frame buffer behavior, and neutral/safe state when focus is lost. **MUST**

FR-012 Combat resolution

Requirement. The game shall resolve attack startup, active/recovery frames, hit/guard/parry, combo scaling, projectile interaction, knockdown, round timer, and win state from data-driven move definitions.

Acceptance. A deterministic test fixture verifies expected frame, damage, stun, meter, and state result for every launch move. **MUST**

FR-013 Ultimate presentation

Requirement. Each fighter shall have one full-Flux ultimate that has authored startup, hit condition, damage, VFX/SFX, and recovery. Its camera/control lock shall not exceed 0.8 seconds.

Acceptance. Test plan confirms each ultimate can be blocked/avoided/punished as specified and begins/ends without desynchronising CombatCore. **MUST**

FR-014 Round and rematch

Requirement. The game shall support best-of-three rounds, timeout result rules, character-select lock-in, rematch, and return-to-menu.

Acceptance. Two local players complete a full match, rematch with same characters, then return without stale health/meter/input state. **MUST**

FR-015 Training lab

Requirement. Training shall provide dummy behavior, reset, frame-step, input display, hitbox overlay, move list, and simple frame advantage readout.

Acceptance. A player can configure block mode, record/play a dummy sequence, toggle hitboxes, and reset in ≤ 2 actions. **SHOULD**

3.3 AI and online play

FR-020 Bot opponent

Requirement. The game shall provide Easy, Normal, and Hard bot settings. Bot choices are generated from deterministic state/seed and constrained by reaction delay and configurable error rates.

Acceptance. Each bot completes 20 simulated matches without invalid input; Easy fails at least 30% of selected punish opportunities in a controlled test. **MUST**

FR-021 Online matchmaking

Requirement. Authenticated players shall enter a casual 1v1 queue by selected region or create/join a room code. Matchmaking shall perform capability/version checks before the loading scene.

Acceptance. Two compatible users join a match; mismatched client version or unavailable region fails before character select with a recoverable message. **MUST**

FR-022 Authoritative online result

Requirement. A match server shall own the authoritative CombatCore timeline, validate input frames, broadcast confirmed state/hash data, and persist only its calculated result. Client-reported damage, wins, or inventory changes shall be ignored.

Acceptance. Modified client attempts to send a fabricated win/damage message; server records no forged result and emits a security event. **MUST**

FR-023 Rollback and recovery

Requirement. Clients shall retain rolling combat snapshots and re-simulate when authoritative/opponent input arrives late. The UI shall distinguish transient correction from a fatal desync.

Acceptance. Under scripted 120 ms RTT, 2% loss and 20 ms jitter, a 5-minute match stays connected and confirmed hashes converge; fatal mismatch exits with a report ID. **MUST**

FR-024 Disconnect policy

Requirement. The server shall pause briefly on a dropped player, permit reconnect with a time-limited signed token, and award a forfeit only after the defined grace period.

Acceptance. Disconnect/reconnect within 20 seconds resumes from a server-confirmed frame; failure to return yields a logged forfeit, not duplicate rewards. **SHOULD**

3.4 Progression, support and telemetry

FR-030 Cosmetic inventory

Requirement. Cosmetic selections shall be server-authorised, presentation-only, and compatible with readability rules (silhouette and team/contrast constraints).

Acceptance. A client cannot equip an unowned item by editing local storage; all approved launch colorways retain minimum contrast in accessibility review. **MUST**

FR-031 Telemetry and diagnostics

Requirement. The game shall record consent-aware technical events: build, browser/GPU class, load milestones, FPS bucket, RTT/jitter, desync, disconnect reason, and anonymised match outcome.

Acceptance. A test session emits documented event schema without raw input text, password, payment, or unnecessary personal data. **MUST**

FR-032 Feedback and reporting

Requirement. Players shall be able to report a player or technical issue after online matches, attaching match ID and optional free text subject to moderation policy.

Acceptance. A report is rate-limited, stored with immutable match reference, and viewable to an authorised support role. **SHOULD**

4. Non-functional requirements

ID	CATEGORY	REQUIREMENT / ACCEPTANCE
NFR-001	Browser support	MUST: Current stable Chrome, Edge, Firefox, and Safari desktop versions; WebGL2 baseline, WebGPU enhancement where available. Detect rather than assume features.
NFR-002	Performance	MUST: 60 FPS target at 1080p on defined mid-range 2020+ desktop test device using Medium preset; if sustained frame time exceeds budget, automatic quality reduction preserves simulation rate.
NFR-003	Load budget	MUST: Critical first-play transfer ≤ 25 MB Brotli-compressed on cold cache; LCP/interactive milestones instrumented. Deferred character/arena cosmetics may stream after menu.
NFR-004	Networking	MUST: Player input is sampled each render frame, sent on fixed cadence with redundancy; server tick is 60 Hz. Design target is acceptable casual play at ≤ 120 ms RTT, $\leq 2\%$ packet loss, ≤ 20 ms jitter.
NFR-005	Availability	SHOULD: Public services target 99.5% monthly availability excluding announced maintenance. Status page and graceful local/bot fallback are required where feasible.
NFR-006	Scalability	MUST: Match capacity scales horizontally by room process/container. Stateless APIs scale independently. A load test proves $2\times$ anticipated concurrent-match capacity before launch.
NFR-007	Security	MUST: HTTPS/WSS only; secure cookies or short-lived tokens; server-side authorisation; schema validation; rate limits; dependency scanning; secrets never shipped in bundle.
NFR-008	Privacy	MUST: Data minimisation, consent-aware analytics, retention schedule, export/deletion workflow, and POPIA-aligned privacy notice before public launch in South Africa.
NFR-009	Accessibility	MUST: remappable inputs, gamepad/keyboard parity, subtitles, separate volume buses, high contrast, reduced flash/shake, no color-only combat information, and focus-visible menus.
NFR-010	Observability	MUST: Sentry-style error reporting, request/match correlation IDs, server health metrics, p95 tick time, queue time, desync rate, and an actionable alert runbook.
NFR-011	Maintainability	MUST: strict TypeScript, lint/format gates, deterministic replay tests, semantic versioned content schema, ADRs for irreversible technical choices.
NFR-012	Localization	SHOULD: all UI strings externalised from day one. English launch; architecture supports later isiZulu, Afrikaans, and other language packs without code changes.

4.1 Performance budgets

AREA	BUDGET / RULE	IMPLEMENTATION LEVER
Frame time	16.67 ms target; allocate ≤ 8 ms render CPU, ≤ 8 ms GPU on Medium.	Batched materials, instancing, LODs, capped dynamic lights, shader variants prewarmed.
Memory	Target ≤ 450 MB resident in a fight; hard warning above 600 MB.	Dispose scene assets on exit; pool projectiles/VFX; avoid duplicate texture decode.
Geometry	$\approx 120k$ visible triangles fight scene at Medium; fighters separately budgeted.	glTF LODs, meshopt/Draco where measured, simple collision proxies.
Textures	Initial arena/fighter texture payload targeted ≤ 18 MB GPU-compressed.	KTX2/Basis Universal; atlases; 1k/2k only where camera proves benefit.
Audio	≤ 20 concurrent SFX voices; music streamed; compressed audio with loop markers.	Web Audio bus manager, priority/ducking, no per-frame allocation.
Network	Input packets compact and redundant; snapshots only as needed for correction.	Binary codec, frame delta encoding, hash every fixed interval; avoid sending transforms each render frame.

COMPATIBILITY BOUNDARY MVP is desktop browser only. Touch controls and low-memory mobile devices are intentionally out of scope until a separate control and performance validation project succeeds.

5. Recommended toolchain and decisions

The recommended stack optimises for a small TypeScript team, true browser delivery, deterministic testability, and a path to quality online play. Pin exact versions in the lockfile at project kickoff; do not paste untested “latest” versions into production.

5.1 Decision table

LAYER	RECOMMENDED TOOL	WHY THIS IS THE BASELINE	ALTERNATIVE / WHEN
Monorepo	pnpm workspaces + Turborepo	Fast installs, shared CombatCore package, cached CI tasks, clear package boundaries.	Nx if the team already relies on its generators and governance.
Language	TypeScript (strict) end to end	One shared deterministic combat implementation across browser and Node server; excellent tooling.	Rust/WASM CombatCore only if profiling proves JS is the bottleneck; adds debugging/build complexity.
Browser shell	Vite + React + Zustand	Fast dev loop, accessible DOM menus, small predictable state model. Keep React out of 60 Hz combat loop.	Solid/Svelte if team proficiency is stronger; avoid large UI frameworks in the canvas loop.
3D runtime	Babylon.js	Mature WebGL2/WebGPU path, glTF loading, cameras, post-processing, GUI integration, inspector tooling. Good fit for 3D browser game.	Three.js offers maximum flexibility but requires assembling more systems; PlayCanvas is strong if a hosted editor workflow is central.
Authoritative combat	Custom shared CombatCore	Exact frame rules, hitboxes, snapshots, and deterministic tests; no engine physics dependency.	Do not make a general 3D physics engine authoritative for fighter outcomes.
Physics polish	Rapier (presentation/non-authoritative)	Fast WASM collision/ragdoll-style utility for props/cosmetics if needed.	Omit entirely if authored collision covers launch needs.
Multiplayer	Node.js + Colyseus rooms + ws, custom fixed-tick input protocol	Rooms, presence and matchmaking scaffolding while retaining control of rollback protocol.	Nakama if the team wants more managed social/leaderboard features and accepts Go/runtime ops; custom ws only if networking expertise is strong.
API & data	Fastify + Zod + Prisma + PostgreSQL	Fast typed REST boundary, schema validation, reliable relational source of truth.	Supabase Auth/Postgres can accelerate authentication/admin; keep game result writes server-owned.
Ephemeral state	Redis	Queues, matchmaking tickets, presence, rate limits, short-lived reconnect reservations.	Managed Redis service to reduce operations.

LAYER	RECOMMENDED TOOL	WHY THIS IS THE BASELINE	ALTERNATIVE / WHEN
Assets	Blender + glTF 2.0 + KTX2 + gltf-transform	Open, browser-native asset path; automate compression/validation.	Use Mixamo only for appropriately licensed prototype animation, then replace/retarget for production.
Audio	Reaper/Audacity + Web Audio / Howler	Small-team practical production and browser mixing.	FMOD/Wwise web workflows only when audio scope and license justify integration.
CI/CD	GitHub Actions + Docker + IaC	Test, build, scan, deploy repeatably.	GitLab CI or existing enterprise pipeline if already operated.
Monitoring	Sentry + OpenTelemetry + Grafana/managed metrics	Errors, client context, traces, capacity/tick dashboards.	Cloud provider-native logs alone are insufficient for gameplay diagnosis.

5.2 Why not build this as a free-roam 3D shooter?

A fully free-camera 3D brawler multiplies camera, aiming, collision, stage, AI, balance, networking, animation, and accessibility cost. The 2.5D format still gives a 3D visual identity, magic, weapons, and mobility, but gives every attack a legible lane and a finite set of counterplay rules. This is the highest-quality choice for a small browser indie team.

5.3 Repository layout

```

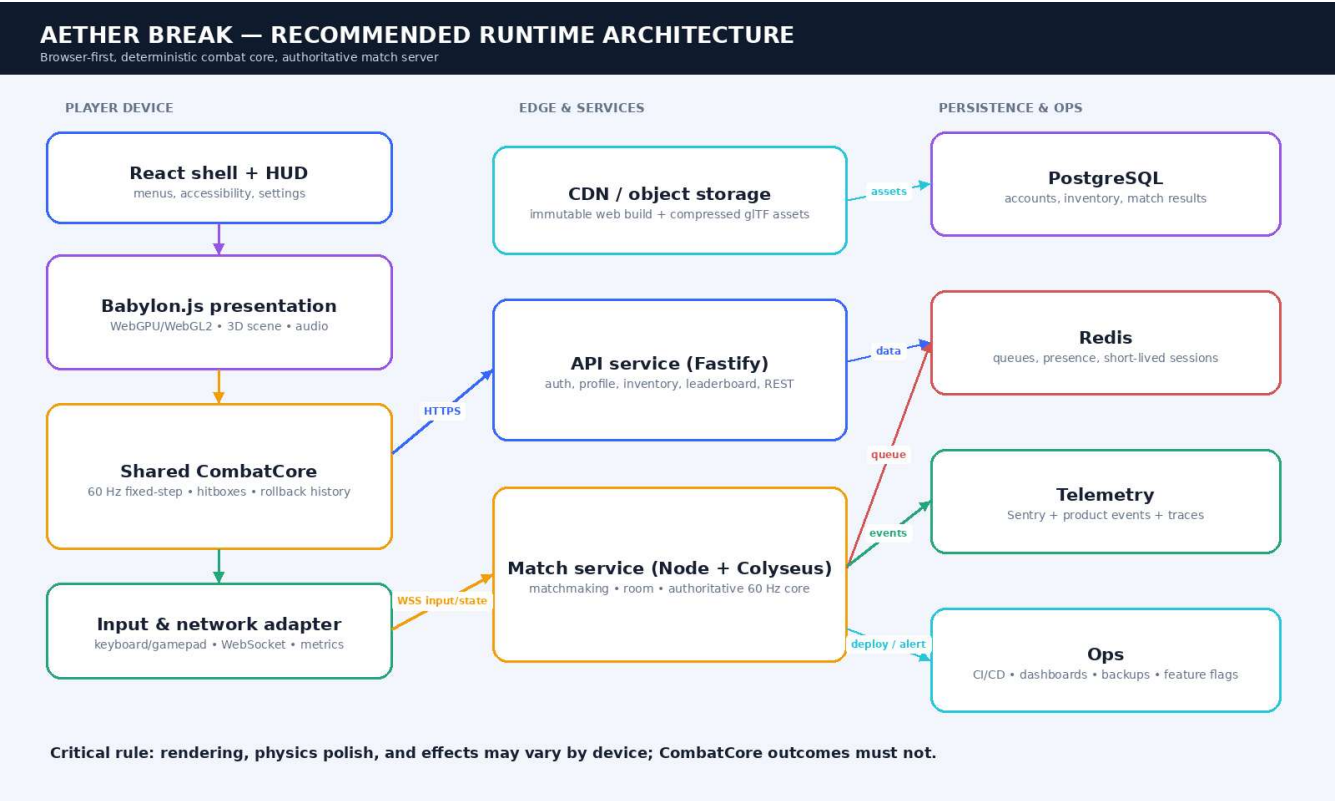
aether-break/
  apps/
    web/           # Vite/React shell, Babylon scene adapters, UI
    api/           # Fastify REST/admin service
    match-server/  # Colyseus rooms, authoritative simulation adapter
  packages/
    combat-core/   # pure deterministic state machine, move data, hashing
    protocol/      # message schemas, binary codec, version negotiation
    content/       # fighters, moves, bot profiles, localization schemas
    ui/            # shared DOM UI primitives and design tokens
    config/        # tsconfig, eslint, vitest defaults
  assets/
    source/        # Blender, audio source; access-controlled in real repo
    processed/     # glTF/KTX2 manifests generated by pipeline
  infra/          # Docker, IaC, dashboards, runbooks
  docs/           # ADRs, balance notes, privacy, support procedures

```

HARD BOUNDARY packages/combat-core may import only deterministic utilities and content data. It may not import Babylon, React, Web Audio, Date, Math.random, fetch, WebSocket, or browser globals.

6. System architecture

The architecture separates authoritative decisions from presentation. A player should feel an immediate local response, but the server confirms the timeline and owns persistence. The same CombatCore code runs in both places to avoid “client rules” and “server rules” drifting apart.



6.2 Match server responsibilities

- Create a room only after client version, content manifest hash, capability, region and authentication checks pass. Lock the content manifest for the whole match.
- Receive input frames with sequence numbers; reject malformed, future/out-of-window, rate-excessive, duplicate or unauthorised messages. Keep a short input history for dispute/replay diagnostics.
- Run authoritative CombatCore at 60 Hz. Broadcast ack/confirmed frame and state hash at a measured interval; send compact resync snapshots only when required.
- On finish, calculate result from authoritative state, write an idempotent match record, then issue cosmetic/progression grants through server-side transaction/outbox.
- Emit metrics, structured logs and a correlation/match ID. Rate-limit join, chat/report (if shipped), and reconnect endpoints. Never embed server admin or database credentials in a web build.

6.3 Online timeline and rollback protocol

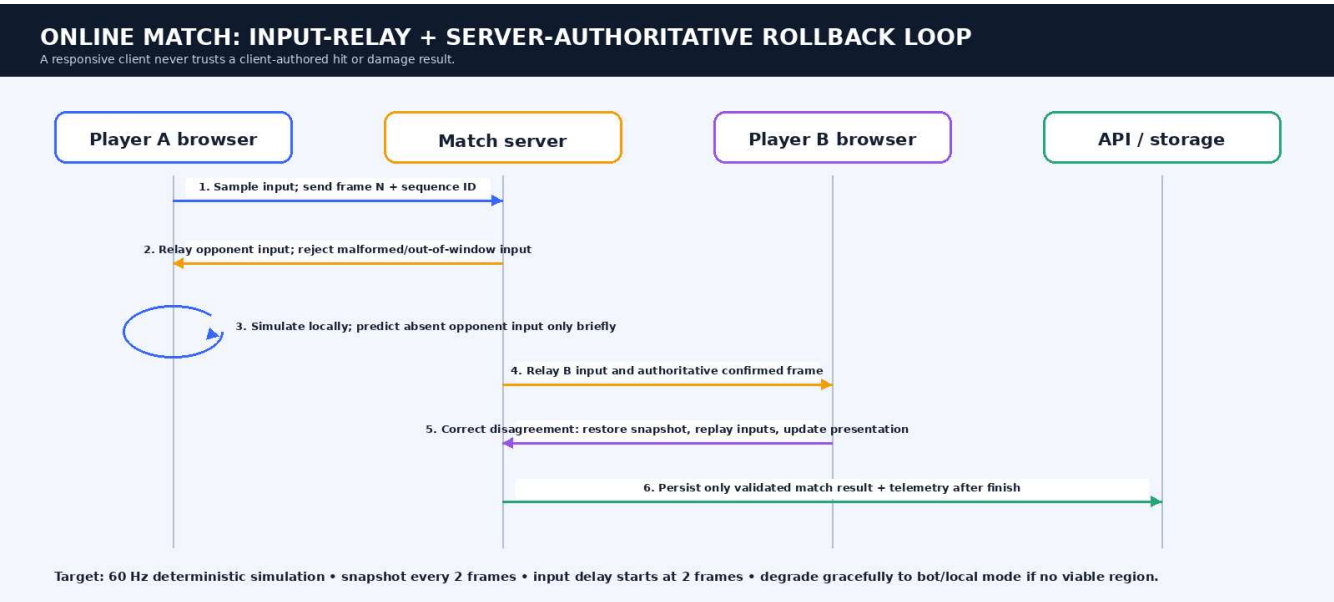


Figure 2. Deterministic input relay, confirmation and correction.

DESIGN CHOICE	SPECIFICATION
Input transport	WebSocket over WSS. Binary, versioned messages: HELLO, JOIN, INPUT, ACK/HASH, SNAPSHOT, PING, RECONNECT, ERROR. One input carries current plus a few prior unacknowledged frames.
Frame protocol	Start with 2-frame local input delay; tune by regional test. Server accepts a bounded future window, buffers both players, predicts briefly only under defined rule, then confirms frames.
Rollback	Store full serialisable CombatCore snapshot every 2 ticks and input history for ≥240 ticks. Late authoritative/opponent input causes restore to prior snapshot and re-simulate forward; renderer smoothly reconciles.
Determinism	Fixed tick, fixed-point/integer positions/velocities, seeded PRNG, stable sort order, tables of moves. Ban locale/time/random APIs. Hash critical state every 30 ticks in dev/test and a measured production cadence.
Desync	If client/server hash mismatch persists after one snapshot correction, halt result issuance, show recoverable error, upload diagnostic metadata, and classify cause. Do not silently pick a client winner.
Latency UX	Show unobtrusive connection indicator and post-match RTT/jitter. Avoid dynamic per-player gameplay advantage; if quality is unacceptable before round start, cancel without penalty.

6.4 Service interfaces and data model

REST is for slow/account operations; WebSocket is for the live match. All external routes have explicit version, authentication and Zod-like validation. Use idempotency keys for result and purchase-like mutations—even if launch has no real-money store.

INTERFACE	EXAMPLE OPERATIONS	AUTH / NOTES
HTTPS /api/v1	POST /session/guest; GET /profile; PATCH /settings; GET /inventory; GET /leaderboard; POST /reports	Short-lived access token; API verifies user ownership. No combat outcome endpoint exposed to client.
WSS /match	HELLO(build, manifest); QUEUE(region); CREATE_ROOM; JOIN(code); INPUT(frame, bits); PING; RECONNECT	Match token is short-lived, room-scoped. Protocol uses server-issued player slot and monotonic sequence.
Internal worker	match.finished; progression.grant; report.created; telemetry.aggregate	Private network/service identity only. Outbox pattern prevents missed rewards after DB failure.
Content CDN	/manifest.json; immutable /assets/{hash}.glb/.ktx2/.opus	Public immutable artifacts; manifest signed/versioned; content hashes verified by client.

ENTITY	KEY FIELDS	OWNERSHIP / RETENTION
UserProfile	id, createdAt, displayName, regionPreference, consentVersion, settingsRef	Authenticated user; minimise PII; deletion/anonymisation workflow.
InventoryItem	userId, cosmeticId, source, acquiredAt, version	Server-owned. Cosmetic IDs refer to read-only manifest.
Match	id, buildHash, contentHash, region, players, start/end, result, disconnectReason, integrityFlags	Server-authored. Keep aggregate/diagnostic period defined in privacy schedule.
MatchInputDiag	matchId, compressed frames/hash samples, expiry	Restricted access; short retention solely for desync/abuse diagnosis.
Report	id, reporterId, matchId, reason, text, status, timestamps	Restricted moderator access; retention per published policy.
FeatureFlag	key, environment, rollout, constraints, audit fields	Operator-owned; no flag may alter competitive move data mid-match.

6.5 Deployment topology

COMPONENT	RECOMMENDED DEPLOYMENT	OPERATING NOTE
Web client/assets	Global CDN + object storage, immutable hashed builds, Brotli compression.	Serve HTML with short cache; versioned JS/assets long cache. Publish source maps privately to error tracker.
API	Container service behind HTTPS load balancer; at least two instances for public launch.	Stateless; migrations run as a controlled job. WAF/rate limits at edge.

COMPONENT	RECOMMENDED DEPLOYMENT	OPERATING NOTE
Match servers	Regional container/VM pool with TCP/WebSocket support and autoscaling on active rooms.	Place initial servers where measured player latency supports the audience; run a South Africa regional probe before selecting production region.
Data	Managed PostgreSQL with encrypted backups and point-in-time recovery; managed Redis with failover plan.	Database is not on the match hot path beyond match start/end and explicit service events.
Secrets/identity	Managed secrets vault + separate service identities per environment.	Rotate credentials; local .env never committed; production access audited.
Environments	dev, shared test, staging, production; separate data/keys/CDNs.	Promotion requires content manifest compatibility and automated smoke test.

INFRASTRUCTURE SPIKE Before locking hosting, run 20-minute two-player probes from intended launch locations. Measure RTT, jitter, WebSocket disconnects, tick time, and egress cost. Select the region using this evidence—not a provider marketing map.

7. Content, art and production pipeline

7.1 Art direction and asset limits

Art direction: high-contrast “arcane industrial” world—painted-metal materials, glowing runes, clean silhouettes, and deliberately chunky readable forms. The game should look premium at a small asset count, not chase photorealism. Character color zones must remain distinguishable against every arena variant.

ASSET	MVP BUDGET / RULE	DELIVERY FORMAT
Fighter	4 launch fighters; 25–35k triangles LOD0 target, LOD1/2 required; shared humanoid baseline where it does not erase silhouette.	Blender source → glTF 2.0; KTX2 textures; material naming convention; generated asset manifest.
Animation	≈45 reusable universal clips + 12–18 fighter-specific actions each; authored timing events by frame.	glTF animation clips; event map externalised in content package, not hand-wired into renderer.
Arena	One collision-simple lane, modular background, 2 lighting variants, ≤120k visible triangles Medium.	Static glTF chunks, baked lighting where appropriate, visibility/LOD groups.
VFX	Pooled particle/mesh effects, <12 concurrent major emitters per player. Color + shape communicate source and state.	Atlas/mesh effects; quality tiers; all emitters disposable and no authoritative dependency.
Audio	One music loop, 4 character voice sets, universal impacts, weapon/spell layers, UI and ambience buses.	Licensed WAV source; browser-ready Opus/AAC as appropriate; loudness and loop metadata.
UI	DOM menus for accessibility; canvas/HUD overlays only where latency/scene position demands it.	Figma tokens → shared UI package; localization keys, not embedded strings.

7.2 Fighter authoring contract

Every attack, projectile, ability and ultimate is content data validated against a schema. Code exposes a small set of tested primitives (spawn projectile, apply impulse, set state, consume meter, create barrier, etc.). Designers compose those primitives; bespoke code requires an ADR and automated regression tests.

```
{
  "fighterId": "nyra_vex", "version": "1.0.0", "base": { "hp": 1000, "walk": 286 },
  "moves": [{
    "id": "arc_light_1", "input": ["LIGHT"], "startup": 6, "active": [6, 8], "recovery": 12,
    "hitboxes": [{ "frame": 6, "shape": "box", "x": 46, "y": 74, "w": 38, "h": 24 }],
    "onHit": { "damage": 42, "hitStun": 15, "fluxGain": 4 },
    "onBlock": { "blockStun": 9, "advantage": -2 }, "cancelTo": ["arc_light_2", "phase_step"]
  }]
}
```

PIPELINE GATE	OWNER	PASS CONDITION
Concept / kit card	Game designer + artist	Role, counterplay, silhouette, input list, and performance risk approved before modelling.

PIPELINE GATE	OWNER	PASS CONDITION
Greybox fighter	Gameplay engineer + designer	All moves exist in data; training frame tests and matchup review complete; no final art required.
Art / animation integration	Animator + technical artist	Clip/event validation passes; root motion policy consistent; LOD/texture/memory budget meets target.
Balance candidate	QA + designer	100+ internal round dataset; exploitable loop test; bot/online replay fixtures pass.
Release candidate	Producer + QA + engineering	Content manifest signed, smoke test in staging, rollback compatibility confirmed, localisation/a11y checked.

7.3 Audio and feedback system

- Use layered feedback: animation anticipation + VFX color/shape + impact sound + hit-stop + controller vibration (where browser API permits). No critical state is communicated by color alone.
- Define buses: Master, Music, SFX, Voice, UI, Ambience. Duck music under ultimates by a modest, tested amount; subtitles capture voiced gameplay callouts when enabled.
- Maintain a source/license register for every texture, sound, font, animation, plugin, and generative/contract asset. “Found online” is not a license.

8. Quality, security and live operations

8.1 Test strategy

TEST LAYER	TOOLS / METHOD	MINIMUM GATE
Combat unit	Vitest; deterministic fixtures for every move, meter, projectile, state transition, parser and hash.	Every launch move has hit, block, whiff, cancel and boundary test; cross-runtime hash suite green.
Property / simulation	Seeded thousands-of-round bot simulations; fuzz invalid inputs and content schema.	No NaN, out-of-bounds state, stuck round, negative resource, or invalid transition across agreed seed set.
Integration	Ephemeral API/match room; protocol contract tests; DB transaction/outbox tests.	Join/queue/match/result/reward/reconnect flows pass against staging-like services.
Browser E2E	Playwright on supported browser matrix; keyboard and gamepad emulation where available.	Guest practice, auth, settings persistence, local match, and failed-network UX pass.
Network soak	Network impairment (RTT/loss/jitter), headless client load, multi-region human sessions.	Defined NFR-004 path converges and diagnostics classify deliberately induced failures.
Performance	Chrome/Firefox profiling, GPU frame capture, asset/memory audit on test hardware tiers.	No release candidate exceeds frame/load/memory budgets without written waiver.
Accessibility	Keyboard-only, contrast, reduced motion, screen-reader menu pass, player test.	No blocker in critical start/match/settings paths; documented exceptions have alternative path.
Security	Dependency scan, SAST, secret scan, authz tests, rate-limit/abuse tests, manual threat review.	No critical/high unmitigated issue; API/client cannot alter authoritative match result.

8.2 Anti-cheat and abuse posture

Browser clients cannot be treated as trusted. The launch anti-cheat strategy is prevention through architecture and detection through telemetry—not invasive client software.

THREAT	CONTROL	DETECTION / RESPONSE
Forged damage/win/reward	Server re-simulates/owns CombatCore and result transaction; client has no result write endpoint.	State hash mismatch and impossible transitions flagged; result withheld on fatal integrity failure.
Input speed / macros	Frame window, input schema, action/state validation, cooldowns in core. Do not ban solely for high skill.	Flag physically implausible patterns for review only; avoid false-positive automatic bans at MVP.

THREAT	CONTROL	DETECTION / RESPONSE
Protocol abuse / bots	WSS auth, nonce/sequence, room capacity, IP/account rate limits, WAF, schema size limits.	Connection and malformed-message counters; temporary rate limit / challenge / operator review.
Client tampering	Build/content manifest hashes and compatibility handshake; server validates actual outcomes.	Version mismatch denied; telemetry records hash/manifest anomalies.
Harassment	No launch voice chat. Display-name rules, report flow, audit trail, moderation runbook.	Rate-limited reports; least-privilege moderator access; documented appeal policy before public launch.

8.3 Privacy, legal and community essentials

- Publish Terms, Privacy Notice, Code of Conduct, support contact, cookie/analytics consent where required, and age/parental approach appropriate to the launch territory before public release. Obtain qualified legal review; this document is not legal advice.
- For a South African launch audience, design data handling around POPIA principles: purpose limitation, minimal collection, security safeguards, retention, access/correction/deletion process, operator agreements, and an accountable owner.
- Maintain third-party notices and licenses for software and assets. Do not market the game using Street Fighter trade dress, copyrighted character likenesses, move names, sounds, or copied animations; “inspired by” is a direction, not a licence.

8.4 Operational playbook

SIGNAL	FIRST RESPONSE	OWNER / SLO
Match server p95 tick >12 ms for 5 min	Stop new rooms on affected pool, scale/shift capacity, preserve logs; do not slow 60 Hz silently.	On-call engineer; acknowledge ≤15 min public launch.
Desync rate rises above baseline	Disable affected content/build flag if safe, collect match IDs, investigate reproducible replay.	Gameplay + backend; triage ≤1 hour.
Authentication/CDN outage	Show status and enable local/practice if static app is reachable; do not loop login retries.	Ops; status update ≤30 min.
Suspected result exploit	Disable progression grants or queue if necessary; retain server diagnostics; communicate limited facts.	Security/product lead; immediate containment.
Critical client crash	Roll back CDN release/feature flag, upload source maps, publish workaround.	Release owner; rollback decision ≤30 min.

9. Delivery plan, risks and launch readiness

9.1 Team assumption and milestones

Indicative plan—not a fixed bid. Assumes a lean team of 4–6: gameplay/client engineer, backend/network engineer (part time early), 3D artist/animator/technical artist, game designer/producer, QA support, and contracted audio/UI support. Reduce scope rather than quality if staffing is lower.

MILESTONE	DURATION	DELIVERABLE / EXIT CRITERIA
0. Pre-production + technical spike	2–3 weeks	Combat paper design, input prototype, browser GPU profile, asset test, two-client deterministic hash proof, hosting latency probes, ADRs.
1. Greybox vertical slice	4–6 weeks	Two greybox fighters, one lane, local/AI duel, 60 Hz CombatCore, 60 FPS target, hit feel review. No online promise until fun is proven.
2. Online + content foundation	5–7 weeks	Authoritative room, rollback/reconnect, four fighter data kits at greybox, training tools, asset pipeline, telemetry and basic profile.
3. Alpha	4–5 weeks	Final-ish four fighter art/animation, arena variants, casual queue/room code, bot pass, performance/content/schema lock, structured external playtest.
4. Beta / release candidate	3–4 weeks	Bug/compatibility/accessibility/security fixes, load/soak test, legal/support pages, runbooks, release rehearsal, no new core mechanics.
5. Soft launch & observe	2 weeks minimum	Limited geography/audience, dashboards watched, queue/latency and crash data reviewed before expansion.

9.2 Prioritised backlog

NOW — MUST FOR VERTICAL SLICE	NEXT — MUST FOR MVP	LATER — ONLY AFTER PROOF
Movement, attack/block/dash, one gun, one spell, hit feedback; two greybox fighters; local versus; deterministic test harness; one target browser performance profile.	Four fighters; online casual; training; bots; profile/cosmetics; art/audio polish; telemetry/monitoring; accessibility and legal baseline; deployment/runbooks.	Ranked ladder, second arena, spectator/replay sharing, more fighters, seasons, story, mobile controls, monetisation, voice chat.

9.3 Risk register

RISK	LIKELIHOOD / IMPACT	MITIGATION & TRIGGER
Online rollback desync	Medium / High	Prove shared 60 Hz hash determinism in milestone 0; keep physics out of core. Trigger: any unexplained cross-runtime divergence blocks content expansion.
Scope creep from “one more character/mode”	High / High	Content budget and change gate. Trigger: a feature lacks owner, test plan, art/CPU/memory budget—move it out of MVP.

RISK	LIKELIHOOD / IMPACT	MITIGATION & TRIGGER
Browser GPU/device variance	Medium / High	Performance tiers, WebGL2 baseline, hardware test matrix, early asset profiling. Trigger: median 60 FPS miss—reduce effects/LOD before adding content.
Poor regional latency	Medium / High	Run real regional probes and show connection quality. Trigger: median RTT/jitter fails target—limit queue regions or delay online launch, retain local/bot value.
Character balance exploit	High / Medium	Data-driven moves, matchup tests, replay telemetry, small roster. Trigger: dominant strategy in test data—hotfix data only with match manifest versioning.
Asset/animation throughput	Medium / Medium	Greybox first, shared rig/pipeline, strict clip list. Trigger: a final fighter misses integration gate—ship fewer polished fighters rather than placeholder quality.
Third-party asset/IP/licensing issue	Low / High	License register and original art direction from day one. Trigger: missing provenance—remove/replace before external test.
Operations cost surprise	Medium / Medium	Load/egress budget and regional probe; autoscale rooms, cap dev environments. Trigger: cost/match exceeds threshold—profile packet rate/assets before scaling marketing.

9.4 Release readiness checklist

- ☐ CombatCore deterministic hash suite passes across server and supported browser matrix; known simulation limits documented.
- ☐ All MUST functional and non-functional requirements have a test record; remaining SHOULD items have explicit disposition.
- ☐ No critical/high unmitigated security issue; abuse, incident, rollback and support owners are named.
- ☐ Performance/load/memory targets pass on agreed hardware and network matrix; quality fallback validated.
- ☐ Online soak/load result supports planned concurrency; dashboard, alerts, status messaging and rollback deployment rehearsed.
- ☐ Accessibility, privacy, terms, third-party notices, age/rating approach and asset licenses reviewed and published.
- ☐ No gameplay-affecting paid item; cosmetic catalog and contrast/readability checks approved.
- ☐ Soft-launch metrics thresholds and go/no-go meeting date are agreed before traffic acquisition.

GO / NO-GO RULE If the game cannot deliver a stable, readable, locally fun two-fighter duel at 60 FPS, do not expand roster, arena count, or monetisation. Fix the core loop first.

Appendix A — Default input map

Every action is remappable. The default avoids mouse aiming so keyboard and gamepad share the same combat language. A player may use keyboard-only or a standard gamepad.

ACTION	KEYBOARD DEFAULT	GAMEPAD DEFAULT	NOTES
Move	A / D; crouch S; jump Space	Left stick / D-pad; jump South button	Left/right orient relative to locked camera.
Light / Heavy	J / K	West / North	Normals and context combo follow-ups.
Ranged	L	East	Gun or character ranged cast; resource governed.
Ability 1 / 2	U / I	Left bumper / Right bumper	Character abilities; cooldown/resource icon always visible.
Guard / Parry	Left Shift	Right trigger	Hold to guard; precise release/press timing enables parry where allowed.
Dash / Sidestep	W + direction / E	Right bumper modifier / right stick tap	Final accessibility-friendly mapping to be tested; no double-tap-only requirement.
Ultimate	O	Both triggers	Only active at full Flux; requires deliberate combination to prevent accidental fire.
Pause / training reset	Esc / R	Menu / View	Online pause governed by match policy; reset is training/local only.

Appendix B — Combat state machine

RoundIntro → Neutral ⇔ { Walk, Crouch, Jump, Dash, Guard, Attack, Ability, Hitstun, Blockstun, Knockdown, Airborne, UltimateStartup, UltimateActive }
Neutral / eligible action → RoundEnd → Result → Rematch | CharacterSelect | Menu

Rules: transitions are validated by CombatCore; presentation subscribes to events. A state may only leave through its authored timer/condition. Hitstop freezes selected animation/presentation clocks but never creates an extra simulation tick. Online correction replays state transitions, not visuals.

Appendix C — Protocol message sketch

MESSAGE	FIELDS (ILLUSTRATIVE)	VALIDATION
---------	-----------------------	------------

MESSAGE	FIELDS (ILLUSTRATIVE)	VALIDATION
HELLO	protocolVersion, buildHash, contentHash, capabilities, authToken	Known supported version/manifest; token verified; no raw device fingerprint required.
INPUT	roomId, playerSlot, frame, sequence, actionBits, redundantFrames[]	Authenticated room slot; monotonically valid sequence/frame window; fixed byte length/action mask.
ACK_HASH	serverFrame, confirmedFrame, ackSequence, stateHash	Server authored only; client uses it for reconciliation/diagnostics.
SNAPSHOT	baseFrame, compressedCombatState, inputTail, reason	Server authored; schema/version/hash checked before client restoration.
MATCH_END	matchId, winnerSlot, reason, finalHash, signature	Server authored only; client displays result; API inventory fetch remains authoritative.

Appendix D — Glossary

TERM	MEANING
2.5D	3D visuals and depth presentation with combat constrained to a readable side-on lane plus limited authored evade depth.
CombatCore	Pure deterministic package containing the authoritative rules, state, move data integration, input processing and state hashing.
Rollback	Restore a saved recent state and re-simulate frames after late input arrives, so the game can remain responsive under latency.
Hitbox / hurtbox	Authored volumes that apply an attack / receive a hit. They are gameplay data, distinct from visual mesh and physics collision.
Frame advantage	How many frames one fighter can act before the other after an interaction; a key training/balance measure.
Manifest hash	Versioned hash of compatible code/content assets, preventing clients with different move data from entering the same match.
Authoritative	The server's decision is final for result, progression, and validated simulation; client display is not trusted.
Soft launch	A controlled public release used to validate stability, retention, support and operations before broader promotion.

Appendix E — References to establish at kickoff

Create living links in the project wiki for tool documentation, browser support policy, licensing register, privacy/legal advice, security policy, accessibility test plan, art bible, move data schema, and architecture decision records. Recommended starting documentation:

[Babylon.js documentation](#) • [WebGPU specification](#) • [glTF 2.0](#) • [Colyseus documentation](#) • [OWASP ASVS](#) • [WCAG 2.2](#)

End of specification.